VELLORE INSTITUTE OF TECHNOLOGY, VELLORE

SCHOOL OF COMPUTER SCIENCE AND ENGINEERING

BCSE307P - COMPILER DESIGN LAB
PROJECT REPORT

CodeTrace: Compiler-Driven Program and Security Analysis for a C Subset

Submitted by

Srikiran - 24BCE2215
Dhruv - 24BCE2211
Prince - 24BDS0153

Under the guidance of

Nithya K,
Assistant Professor Senior Grade II

OCTOBER, 2026

BONAFIDE CERTIFICATE

This is to certify that the project report titled *CodeTrace: Compiler-Driven Program and Security Analysis for a C Subset* is a bonafide record of work carried out by Srikiran (24BCE2215), Dhruv (24BCE2211) and Prince (24BDS0153), submitted in partial fulfilment of the requirements for the course BCSE307P - Compiler Design Lab, School of Computer Science and Engineering, Vellore Institute of Technology, Vellore, during the academic year 2026 - 2027.

The report documents the implementation present in the project repository on the date of submission. Every measurement quoted in the report was produced by the commands named in Chapter 4, on that checkout, and is reproduced by the generation script rather than transcribed by hand.

Faculty Guide: _____

Date: _____

DECLARATION

We hereby declare that the project report titled *CodeTrace: Compiler-Driven Program and Security Analysis for a C Subset*, submitted by us to Vellore Institute of Technology, Vellore in partial fulfilment of the requirement for the course BCSE307P - Compiler Design Lab, is a record of bona fide work carried out by us under the supervision of Nithya K. The contents of this report, in full or in part, have not been submitted to any other institute or university for the award of any degree or diploma.

The source code, test suites, example corpus and measurement scripts described in this report are part of the project repository, and this document is generated from its maintained source rather than assembled by hand.

Place: _____

Date: _____

Signature of the candidate(s): _____

ABSTRACT

CodeTrace is a static program-analysis platform for CT-C, a small and fully documented subset of C. It joins compiler construction to program-dependency and security analysis in a single inspectable workflow: a hand-written lexer produces located tokens, a canonical LR(1) parser generator builds SLR(1), LALR(1) and canonical LR(1) tables from the project's own grammar, and semantic analysis resolves declarations into symbol identifiers that every later stage uses instead of names. Per-function control-flow graphs are then constructed and a shared worklist framework computes reaching definitions, live variables and definite assignment. A declarative rule language, parsed by the same LR machinery, declares taint sources, sinks, context-specific sanitisers and validator guards. The taint engine is a flow-sensitive may-analysis over taint atoms, with context-insensitive function summaries for interprocedural propagation, and it records why a sink call was *not* reported when that is the case. A typed dependency graph and backward, forward and finding-focused slices expose the statements related to each result, while JSON and self-contained HTML reports support review outside the browser. The analysed program is never executed. On this checkout the engine suite reports 350 passing tests, the interface suite 53, the repository audit 69 passing checks, and the 33-file labelled corpus 20 true positives with no false positives or false negatives. The contribution is educational and architectural rather than competitive: the compiler representations and the analysis reasoning stay visible beside the security answer, inside a language subset whose boundaries are stated explicitly.

ABSTRACT **4**
CHAPTER 1 INTRODUCTION **6**
1.1 Background 6
1.2 Problem Statement 6
1.3 Objectives and Contributions 7
1.4 Scope, Terminology and Reading Guide 8
CHAPTER 2 LITERATURE SURVEY **9**
2.1 Existing Systems and Related Work 9
2.2 Research Gap 10
CHAPTER 3 SYSTEM DESIGN **11**
3.1 Proposed System Architecture 11
3.2 Module Description 12
3.3 Compiler Phases and Analysis Pipeline 13
CHAPTER 4 IMPLEMENTATION DETAILS **18**
4.1 Technology and Application Workflow 18
4.2 Worked Example 18
4.3 Testing and Verification Evidence 20
4.4 Current Progress and Challenges 22
CONCLUSION AND FUTURE ENHANCEMENTS **23**
REFERENCES **24**

CHAPTER 1
INTRODUCTION

1.1 Background

A compiler front end turns source text into a structured representation that later stages can reason about. Lexical analysis groups characters into tokens, syntax analysis decides whether the token sequence follows a grammar, and semantic analysis resolves declarations and checks the relationships between names, types and operations [1]. These stages are usually taught as preparation for code generation, but they are equally the foundation of program analysis: once a program is represented as an abstract syntax tree with resolved identifiers, an analysis can ask how values move through statements, branches and function calls.

Two classical families of analysis build on that foundation. Data-flow analysis computes facts at program points by following control flow, and its standard formulation - a finite lattice, monotone transfer functions and a fixed point reached by iteration - covers reaching definitions, live variables and definite assignment [4, 5, 6]. Program-dependence analysis records the data and control relationships between statements, and program slicing uses those relationships to extract the statements that can influence a chosen point [7, 8, 9]. Both families assume a control-flow graph and resolved names; neither can be built directly on raw source text without risking confusion between two variables that happen to share a spelling.

Security-oriented static analysis applies the same machinery to a different question: can data introduced at an untrusted source reach an operation that requires particular handling? Injection weaknesses - command injection, SQL injection, path traversal, format-string misuse - are the canonical examples, catalogued as CWE-78, CWE-89, CWE-22 and CWE-134 [16]. Practical tools of this kind combine a source specification, a sink specification and a propagation analysis, and their usefulness depends as much on the explanations they produce as on the flows they detect [11, 12, 13]. Studies of developer behaviour point in the same direction: warnings that cannot be understood, acted on or trusted are ignored, and adoption depends on how well a tool explains itself inside an ordinary workflow [14, 15].

CodeTrace applies these ideas to CT-C, a documented subset of C with scalar types, fixed-size arrays, functions, conditionals and loops. The front end, the intermediate representations and the analysis results are all exposed for inspection through a browser interface and an HTTP API. The intent is not to compete with a production static application security testing tool, and not to accept full C: it is to show, in one working system, how compiler-design components support a traceable security analysis whose assumptions remain visible.

1.2 Problem Statement

A security finding is difficult to evaluate when a tool reports only a rule name and a line number. Without a model of syntax, symbol identity and control flow, a reader cannot tell whether a value can actually reach a sensitive operation, which assignments and calls connect the two, or where an analysis assumption weakens the conclusion. A finding that says "command injection at line 10" leaves the interesting part - the path from input to sink, and the reason the path was not excluded - entirely implicit.

A second problem is the interpretation of an empty result. The absence of a reported flow is meaningful only in relation to the constructs, rules and approximations that were actually considered. If a file uses a construct the analyser does not model, a clean finding list is not evidence of safety. A tool that does not state its boundaries invites a reader to mistake silence for proof.

A third problem is educational. Compiler courses teach lexing, parsing, semantic analysis, intermediate representations and data-flow analysis as separate topics, often with separate small exercises. Students can implement each stage and still find it difficult to explain how the stages compose into an analysis of a whole program, or why a particular analysis result follows from a particular representation. An integrated artefact that keeps the representations inspectable closes that gap.

The project therefore addresses the following problem: given a single source file in the supported CT-C subset, construct the compiler representations needed for analysis, derive selected control-flow and data-flow facts, identify potential flows from rule-declared sources to rule-declared sinks, and present the reasoning and the limits behind each result. The analysed source must remain data throughout; it is never executed as part of the analysis. The implementation must additionally run offline, use only the Python standard library and a dependency-free browser interface, and bound its own work so that a pathological input cannot exhaust the process.

1.3 Objectives and Contributions

The objectives of the work are:

- Implement a lexer that preserves source positions, reports located diagnostics for malformed input and distinguishes constructs outside the supported subset from ordinary syntax errors.
- Generate LR-family parsing tables from the CT-C grammar, provide canonical LR(1), LALR(1) and SLR(1) modes, and build a source-located abstract syntax tree with grammar-driven semantic actions.
- Resolve declarations with scoped symbol tables that give every declared name a stable identity, and report semantic problems that affect the reliability of later analysis.
- Construct per-function control-flow graphs with typed edges, and compute reaching definitions, live variables and definite assignment through one reusable fixed-point framework.
- Specify sources, sinks, sanitisers, validators and flows in a declarative rule language that is parsed and validated before use, and propagate taint through assignments, expressions, arguments and return values.
- Build a typed program dependency graph and compute backward, forward and finding-focused slices from it.
- Explain each finding with an ordered evidence path, a certainty level, the applicable limitations and a remediation hint, and record why inspected sink calls produced no finding.
- Expose every artefact through an HTTP API and a browser interface, and produce stable JSON and self-contained HTML reports.

The contributions follow from the way these objectives are met. First, the project integrates established techniques into one pipeline in which every stage consumes an explicit data structure produced by the previous stage, rather than searching the source text again. Second, the LR generator is a general component: the grammar of CT-C and the grammar of the rule language are both compiled to tables by the same code, so the parser generator is demonstrated rather than assumed. Third, sanitisation is modelled per context: an atom carries the context in which a value is unsafe, so escaping a value for

SQL does not silently clear its command-context taint. Fourth, the analysis reports its own silences - a sink call that was inspected and produced no finding is listed with a reason, and a validator guard that cleared atoms leaves a record naming the guard, the symbol and the context. Fifth, the whole system is reproducible without a package manager: the standard library, a text editor and a browser are sufficient to build, run, test and regenerate this document.

The scope is deliberately bounded. CT-C supports the six scalar types of the subset, fixed-size array declarations, the usual arithmetic, relational and logical operators, functions with parameters and return values, blocks, `if`/`else`, `while`, `for`, `break`, `continue` and `return`. Pointers, structures, unions, enumerations, the preprocessor, `switch`, `goto`, `do`/`while`, multiple translation units and dynamic memory are outside the implemented language model, and each is diagnosed rather than approximated.

1.4 Scope, Terminology and Reading Guide

Three terms are used throughout with a precise meaning. A *finding* is a reported potential source-to-sink flow with its evidence; it is not a claim of exploitability. *Certainty* is the strength of the evidence recorded for a finding and takes one of three values: `confirmed` when the trace was reconstructed without an approximation, `approximate` when a documented approximation contributed to it, and `unproven` when the path could not be justified from the analysis state. A *partial* analysis is one in which the parser recovered from a syntax error, a construct outside the subset was skipped, a semantic error prevented resolution, or a hard limit was reached; a partial run still produces results, but it states that they are incomplete.

Chapter 2 places the work against related literature and identifies the gap. Chapter 3 describes the architecture and the analysis pipeline. Chapter 4 documents the implementation, works through one example end to end, reports the verification actually performed on this checkout, and states the remaining limitations. The conclusion summarises the result and lists realistic extensions.

CHAPTER 2
LITERATURE SURVEY

2.1 Existing Systems and Related Work

Compiler construction supplies the front-end methods used here. The standard treatment of lexical analysis, LR parsing, syntax-directed translation and semantic analysis remains the dragon book [1]. Knuth introduced the left-to-right construction of parsing tables for LR(k) grammars [2], and DeRemer's simple LR method produced a more compact table-construction technique that made LR parsing practical in modest tools [3]. CodeTrace implements the canonical LR(1) item-set construction, the LALR(1) merge of canonical states by kernel, and SLR(1) tables over FOLLOW sets, over a single grammar of 102 productions. The three modes accept the same language because the grammar was written to be conflict-free; they differ in table size, which the interface reports.

Data-flow analysis provides the framework for computing program facts across a control-flow graph. Kildall unified several global optimisations as fixed-point iteration over a lattice with monotone transfer functions [4]; Cousot and Cousot generalised the idea into abstract interpretation, where a program is interpreted over an abstract domain that is a sound approximation of its concrete semantics [5]; Nielson, Nielson and Hankin give the systematic treatment of lattices, meets, transfer functions and worklist algorithms that the implementation follows [6]. CodeTrace uses one worklist solver parameterised by direction, domain, boundary condition, transfer function and meet operator, and instantiates it three times: forward may-analysis for reaching definitions, backward may-analysis for live variables, and forward must-analysis for definite assignment.

Program slicing addresses comprehension: given a criterion, which statements can affect it? Weiser introduced the slicing criterion and the data-flow formulation [7]. Ferrante, Ottenstein and Warren introduced the program dependence graph and showed that slices are reachability problems over data and control dependence edges [8]. Horwitz, Reps and Binkley extended the construction to whole programs with the system dependence graph, whose summary edges make interprocedural slicing precise with respect to calling context [9]. CodeTrace builds a typed dependency graph whose edges carry the analysis result that justified them, and computes backward, forward and finding-focused slices by traversal. It does not build summary edges, so its slices are intraprocedural in their traversal even when a finding crosses a function boundary.

Interprocedural data-flow analysis has a precise general solution. The IFDS framework of Reps, Horwitz and Sagiv reduces distributive interprocedural data-flow problems to graph reachability, obtaining precise results under explicit assumptions [10]. CodeTrace does not implement that algorithm. It computes context-insensitive function summaries - which parameters can influence a function's return value - iterated to a fixed point so that recursion terminates, and it labels every finding that depends on a summary as approximate. The distinction is stated because it explains both the scalability of the implementation and the class of false positive it can produce.

Security-oriented static analysis is the application domain. Chess and McGraw describe static analysis as a scalable way to find security defects before deployment while emphasising that tool output must be actionable and that false positives erode trust [11]. Livshits and Lam show a source-to-sink analysis for Java web applications in which vulnerability specifications are compiled into analyses over a context-sensitive points-to result [12]. Schwartz, Avgerinos and Brumley formalise dynamic taint analysis and forward symbolic execution and discuss the design choices - what taint propagates through, how sanitisation is recognised, how implicit flows are treated - that determine what a taint

analysis can conclude [13]. CodeTrace adopts the source-to-sink model with taint atoms and declared sanitisers, but statically and over a deliberately small language, and it prints the assumptions that a dynamic tool would encode in its instrumentation.

Finally, the literature on tool use informs the presentation. Johnson et al. find that developers avoid static analysis tools whose warnings are hard to understand, hard to act on or untrustworthy [14]; Sadowski et al. report that the successful deployment of analyses at Google depended on workflow integration, low false-positive rates and clear, actionable messages [15]. These findings motivate two design choices in CodeTrace: every finding carries an explanation, a limitation list and a remediation hint, and every run states what it did not analyse.

2.2 Research Gap

The gap addressed by this project is one of integration and inspectability in a compiler-design context rather than a gap in the underlying algorithms. Textbooks explain each algorithm separately, and mature security analysers present conclusions through their own abstractions; a reader who wants to connect the two must usually reconstruct the missing stages. CodeTrace places the lexer, the parsing tables, the syntax tree, the scoped symbol table, the control-flow graph, the data-flow facts, the taint propagation and the dependency graph in one workflow over the same source program, and the browser interface exposes each of them.

A second concern is the interpretation of negative results. A list with no findings does not establish that the source is safe. The implementation therefore records which constructs were unsupported, marks a run as partial whenever a diagnostic or a limit affects the result, and reports, for every sink call it inspected, whether taint failed to reach it, whether the branch was statically pruned, or whether a declared validator guard cleared the flow. Making the boundary of the analysis visible is a design requirement, not a presentation detail.

A third concern is explanation. A useful finding should name the source and the sink, show the sequence of data-flow steps, identify the sanitisers that were seen and did not apply, state the certainty and the limitations, and suggest a direction for repair. The dependency graph and the slice provide a second, structural view of the same result: the evidence path describes the flow, while the graph and the slice show the statements and relationships around it and the reason each statement is included.

Positioning the system honestly also defines its limits. CodeTrace does not attempt whole-program alias analysis, it does not model implicit flows, and its interprocedural summaries are context-insensitive, so its findings are potential flows within a subset and its clean runs are bounded statements about that subset. Those boundaries are the most informative part of the comparison with mature tools, and they define the future work in the concluding chapter.

CHAPTER 3
SYSTEM DESIGN

3.1 Proposed System Architecture

CodeTrace runs as two processes. A Python HTTP server owns the analysis engine, the rule set and a small in-memory store of recent runs; a browser loads a static interface and talks to the server over JSON. There is no database, no build step for the interface, no bundler and no network access beyond the server itself. The request path is:

1. The browser loads `index.html` and the ES modules under `frontend/js` from the same server.
2. The user submits CT-C source, optionally with a rule file, a parser mode and flags that enable or disable the dependency graph.
3. The server validates the request envelope before doing any work: body size, source size, rule size, parameter names, parser mode and rule syntax.
4. The rule file, if supplied, is parsed by the rule-language grammar and validated; errors are returned with line and column, and the run is refused.
5. `analyse_source` runs the pipeline: lexing, LR parsing, semantic analysis, statement numbering, control-flow graph construction, the three data-flow analyses, taint analysis, dependency graph construction, slicing setup and finding construction.
6. The resulting `Program` object is stored under a run identifier in an LRU store of 24 entries.
7. The server returns a summary, the findings, the diagnostics, the grammar and rule statistics, and per-phase timings.
8. The interface renders the run from further endpoint calls: findings with their evidence, the AST, a control-flow graph, data-flow states, the dependency graph, slices and the exported report.
9. Reports are generated on demand from the stored `Program`; neither the JSON nor the HTML report is written to disk by the server.

Two boundaries are enforced in the code. The interface performs no analysis: `frontend/js/graph.js` computes a layout for the nodes it receives and `frontend/js/lib/filter.js` filters the finding array it receives, but no taint, dependency or data-flow value is derived in JavaScript. Conversely, the engine serves structured JSON for the interactive views and only the reporting module renders HTML for the analysed text, with every analysed string escaped.

The `Program` object is the unit of exchange inside the engine. It holds the parse outcome (tokens, tree, accepted flag, timings), the semantic result (symbols, scopes, diagnostics, per-statement accesses), the per-function control-flow graphs, the three data-flow results, the taint result (atoms, hits, summaries, guard suppressions, notes), the dependency graph, the slice engine, the findings, every diagnostic with code, phase, line and column, the measured metrics, the limits that were hit, and the rule set that was actually used. Because the whole object is retained, any later endpoint can answer from the original analysis instead of re-running it.

3.2 Module Description

3.2.1 Language front end

The `codetrace/frontend` package contains the token vocabulary, the hand-written lexer, the grammar representation with NULLABLE, FIRST and FOLLOW computation, the LR(0) and canonical LR(1) automaton builders, the SLR/LALR/canonical table builders and the parsing driver, the AST node definitions, the CT-C grammar with its semantic actions, a source-line index and the `parse_source` entry point. Its outputs are a token stream with offsets and line-and-column positions, a parse tree of AST nodes whose every node carries a source span, a list of syntax diagnostics and a statistics record with parse steps and recovered errors. Parser tables are cached per mode because building them costs more than several analyses.

3.2.2 Semantic analysis

The `codetrace/semantics` package builds scoped symbol tables, assigns every declaration a stable integer identifier, resolves every identifier use to a symbol, records each read and write with its source offset, applies the type rules of the subset, and resolves calls to user functions, declared externals or unknown names. Its outputs are the symbol table, semantic diagnostics with codes such as `SEM001` for an undeclared identifier or `SEM009` for a wrong argument count, and a per-statement access record that the data-flow and taint analyses consume.

3.2.3 Control flow and data flow

`codetrace/analysis/cfg.py` derives a control-flow graph from the shape of the AST, emitting one basic block per maximal straight-line sequence and typed edges for fall-through, true and false branches, loop back edges, `break`, `continue`, return and function exit. It also numbers definitions - assignments, initialised declarations, parameters and increment or decrement operations - so that reaching definitions and the taint engine share one notion of a definition. `dataflow.py` provides the domain, problem and worklist solver; `reaching_definitions.py`, `liveness.py` and `definite_assignment.py` instantiate it. `accesses.py` computes the reads and writes of an individual statement.

3.2.4 Rules and taint analysis

The `codetrace/rules` package defines the rule model (source, sink, sanitiser and flow rules with identifiers, contexts, severities and descriptions), the grammar and parser of the rule language, declarative validation with line-and-column diagnostics, and the built-in rule file written in that language. The taint engine in `codetrace/analysis/taint.py` consumes the parsed rule objects and never evaluates rule text. Its outputs are taint atoms per statement and per symbol, sink hits, context-insensitive function summaries, a convergence flag, guard suppressions and human-readable notes.

3.2.5 Dependencies and slicing

`codetrace/analysis/dependencies.py` builds a typed graph whose nodes are statements, definitions and functions, and whose edges are data, control, call, return, taint, sanitise, define and source-to-sink relations. Every edge stores the analysis result that justified it, so an edge can be displayed with a reason rather than a bare arrow. `slicing.py` traverses that graph backwards, forwards or from a finding, and records for each included statement the reason it belongs to the slice.

3.2.6 Findings and explanation

`codetrace/findings/models.py` defines the finding record: a deterministic identifier derived from the rule, the sink operation, the sink statement text and the contributing source rules; the source and sink records with line, column, statement id, enclosing function and block; the ordered evidence path; the branch conditions; the sanitisers seen; the certainty and confidence; the limitations; and the remediation hint. `explain.py` reconstructs the path from the data-flow and taint states, choosing the shortest justified explanation, and assembles the explanation text, certainty and limitation list.

3.2.7 Reporting, API and interface

`codetrace/reporting/json_report.py` builds the canonical report dictionary, including a section that lists unreported sink calls with a reason for each; `html_report.py` renders the same dictionary as a self-contained document with all analysed text escaped. `codetrace/api/server.py` implements routing, request validation, size limits, the error envelope and the run store; `schemas.py` documents the request and response shapes as data so that the schema endpoint and the tests cannot drift from the implementation. The interface under `frontend/` is a static shell, a stylesheet and ES modules organised one per page plus shared helpers for the API client, state, DOM construction and graph layout.

3.2.8 The browser interface

The interface has nine routes: Overview, Analyze Code, Findings, Program Graph, Data-Flow Analysis, Program Slicing, Rule Editor, Reports and Documentation. Each renders engine output. Selecting a statement in the graph, data-flow or slicing pages fetches that statement's reaching definitions, live variables, taint atoms and control dependencies from the statement endpoint, so the three pages share one renderer and one source of truth. The Documentation page reads the language description, the diagnostic catalogue and the grammar statistics from the API rather than from a copy, which prevents the interface from claiming more than the engine implements.

3.3 Compiler Phases and Analysis Pipeline

3.3.1 Lexical analysis

The lexer is a hand-written deterministic scanner. It performs maximal munch over identifiers, keywords, integer, float, character and string literals, punctuation and operators; it skips line and block comments without emitting tokens; and it records for every token the start and end offset together with the line and column of its first character. Escape sequences in character and string literals are decoded, including `\xHH` forms, and malformed literals produce dedicated codes: `LEX001` for an unterminated literal, `LEX002` for an unterminated block comment, `LEX003` and `LEX004` for malformed literals, `LEX005` and `LEX006` for unknown and malformed escapes, and `LEX007` for a multi-character character constant.

Two design points matter for the analysis that follows. First, keywords that belong to C but not to CT-C are tokenised as a separate kind and reported with `LEX100`, and preprocessor directives with `LEX101`; operators outside the subset such as `->` and `&` produce `LEX102`. These are not generic syntax errors: they mark a construct the analyser deliberately does not model, and they make the run partial. Second, the scanner always advances and reports at most one diagnostic per resynchronisation, so a single run can report several unrelated lexical problems rather than stopping at the first.

3.3.2 Grammar analysis and LR table construction

CT-C subset v1 is defined by a context-free grammar of 102 productions over 51 terminals and 36 non-terminals, including the augmented start production. Precedence and associativity are encoded as grammar levels - assignment, logical or, logical and, equality, relational, additive, multiplicative, unary, postfix and primary - rather than as a precedence table applied during conflict resolution, because the purpose of the project is to build LR tables from a grammar rather than to patch them afterwards.

Two constructions were chosen so that the grammar is conflict-free by construction. The statement grammar is factored into matched and open statements, the standard treatment of the dangling-else ambiguity, so that the grammar does not rely on a default shift preference to resolve it. The `for` loop uses explicit optional non-terminals for its setup, condition and update, which removes the shift/reduce conflict produced by a naive factoring. On this checkout the tables are: SLR(1) with 167 states and 0 conflicts, LALR(1) with 167 states and 0 conflicts, and canonical LR(1) with 656 states and 0 conflicts. The LALR tables contain 2,172 action entries and 382 goto entries.

The automaton construction computes NULLABLE, FIRST and FOLLOW sets by fixed-point iteration over the production list, builds LR(0) item sets and their GOTO transitions, and for canonical LR(1) builds item sets whose items carry lookahead terminals. LALR(1) merges canonical states with identical kernels and unions their lookaheads. Table construction records every conflict with both candidate actions and the resolution applied - shift preferred over reduce, lower-numbered production preferred for reduce/reduce - and the conflict list is available through the API, so a change to the grammar that introduces a conflict is visible rather than silent. The same primitives compile the rule language: 40 productions, 31 terminals, 13 non-terminals, 78 LALR states and 0 conflicts.

3.3.3 Parsing, error recovery and AST construction

The parsing driver is table-driven and iterative: it maintains a state stack and a value stack, shifts and reduces according to ACTION and GOTO, and calls the semantic action attached to a production when it reduces. Each action receives the values of the right-hand side and a span, and returns an AST node or an intermediate value; spans are computed from the first and last child and, for an empty production, from the lookahead token. Every AST node therefore carries a half-open source range, and every later stage can map a result back to text.

Errors are handled by the standard pair of mechanisms. The parser reports `SYN001` with the offending token and the list of terminals that would have been accepted, and then attempts recovery: a panic-mode step that skips tokens until a designated synchronisation token or a token that follows a type keyword, followed by a viable-prefix test that checks whether the current lookahead can be shifted by some suffix of the state stack. A recovery that would make no progress is not attempted, so recovery terminates. Parse steps are capped (`SYN010`) and the number of recovered errors is capped (`SYN011`); either cap marks the run partial. The parse result reports whether the input was accepted, and acceptance means "parsed without recovery", which is the stronger and more useful statement.

3.3.4 Semantic analysis

Semantic analysis runs in two passes. The declaration pass processes every top-level declaration first, giving functions their signatures and global variables their scope, which permits a call to a function defined later in the file without prototypes. The body pass walks function bodies in source order with an explicit scope stack, creating a new scope for each block. Each declaration receives a fresh symbol identifier; each use is resolved through the scope chain and recorded as a read, a write or both, with its offset and, once statement numbering exists, its statement identifier.

The result distinguishes three situations that an analysis must not conflate: a valid construct whose type is known; a valid construct whose value is unknown, such as the result of an unmodelled external call; and a construct outside the subset, which produces a diagnostic with `unsupported` set and marks the enclosing symbol as partial. Type checking covers declarations, assignments, conditions, arithmetic and relational operands, call arguments and return values, with dedicated codes for an undeclared identifier (`SEM001`), a duplicate declaration (`SEM002`), a type error (`SEM005`), an unknown external (`SEM008`), a wrong argument count (`SEM009`), an argument type mismatch (`SEM010`), `break` or `continue` outside a loop (`SEM012`), indexing a non-array (`SEM013`), an indirect call (`SEM017`), shadowing (`SEM018`, a warning) and a function without a return (`SEM020`). Because symbols are identified by number rather than by name, a variable that shadows another in an inner scope is a different symbol everywhere downstream, which is what prevents the taint analysis from confusing them.

3.3.5 Control-flow graph construction

The control-flow graph is derived from the structure of the AST, not from the order of lines in the file. The builder emits one basic block for each maximal straight-line sequence of statements and connects blocks with typed edges: `sequential` for fall-through, `true` and `false` for the two outcomes of a condition, `loop-back` from the end of a loop body to its header, `break` and `continue` for the corresponding jumps, `return` from a return statement to the exit block, and `exit` for a function that falls off its end. Reachability is computed from the entry block rather than assumed, and unreachable blocks are counted and shown.

Statement identifiers are assigned in source order, and definitions are numbered in access order with a kind that records how the value is produced: assignment, initialised declaration, uninitialised declaration, parameter, increment or decrement, and file-scope initialisation. A pseudo statement identifier represents the definitions in force before the first statement of a function, which is where parameters and file-scope variables enter the data-flow lattices.

3.3.6 The data-flow framework

The framework is a worklist solver over a finite lattice. Facts are sets represented as Python integers used as bit vectors, so union and intersection are machine-word operations and the subset test is a single mask comparison. A problem supplies five things: a direction, a domain of atoms, a boundary value, a transfer function per basic block and a meet operator. The solver initialises the boundary, places the entry or exit block on the worklist, and repeatedly applies the transfer function to a block's input, meets the result into the block's output and re-enqueues successors whose input changed. Iterations and processed steps are counted, and both are capped at 10,000 iterations and 200,000 steps; reaching a cap sets `converged` to false and marks the run partial.

Reaching definitions is a forward may-analysis. A definition kills previous definitions of the same symbol and generates itself, the meet is union, and the result is used to justify each data-flow hop in a finding's evidence path and to build data-dependence edges. Live variables is a backward may-analysis over uses and definitions, with intersection at the meet, and it answers which variables may be read later. Definite assignment is a forward must-analysis: a variable is definitely assigned at a point only if every path from entry assigns it, which is what allows the engine to report a read of a possibly uninitialised variable (`DFA001`) without reporting a variable that was assigned before a loop. All three are instantiated from the same solver, which is the practical demonstration that the framework is a framework and not three separate programs.

3.3.7 Taint analysis

The taint state is a mapping from symbol identifier to a set of taint atoms. An atom is a pair of an origin and a context written as text: `src:<rule>@<context>` for data introduced by a configured source, `param:<function>.<index>@<context>` for a parameter during summary computation, and `glob:<name>@<context>` for a file-scope variable. The six contexts are `command`, `sql`, `path`, `html`, `numeric` and `format`. A source rule declares the contexts its data is unsafe in; a sanitiser removes atoms in the context it declares, or in every context when it declares `*`; a validator is a sanitiser used as a branch predicate. This is why escaping a value for SQL does not clear its command-context taint, and it is the main semantic difference between this design and a single-boolean taint flag.

The analysis is a may-analysis: the meet is union, so a value is tainted if any path could taint it, and within a block the transfer function replays statements in order from the block's input state. Calls are handled with context-insensitive summaries. For each function the engine first runs the analysis with every parameter tainted by its own symbolic atom and records which parameters can reach a return statement, iterating to a fixed point so that recursion terminates. At a call site the taint of the result is the union of the taint of the arguments whose positions appear in the summary. A call to a function that is neither defined in the file nor declared in the rule set is unknown: its result is tainted when any argument is tainted, and it is never assumed to validate anything.

Branches are handled with a small amount of path reasoning. An edge guarded by a literal `false`, by a comparison that the analyser can decide, or by a declared validator predicate on a symbol clears the corresponding atoms on that edge, and the cleared atoms are recorded rather than discarded: the suppression records the guard, the symbol identifier, the context, the validator rule, the source position and the removed atoms. Implicit flows are not modelled - a value chosen by a branch on untrusted data is not tainted by the condition - and the finding text and the limitation list say so.

3.3.8 Dependency graph and slicing

The dependency graph has three node kinds: statements, addressed by statement identifier; definitions, addressed by definition identifier; and functions. Its edges carry a kind and a reason. A `data` edge exists when a definition is in the reaching-definitions set of a statement that reads the symbol, and the reason quotes the definition and the read. A `control` edge links a branch or loop condition to the statements nested inside it, derived from the guard stack of the control-flow graph. A `call` edge links a statement containing a call to a function defined in the file; a `return` edge links a callee's return statement to the call site. A `taint` edge marks a data edge that carries at least one atom. A `sanitize` edge marks a statement that applies a declared sanitiser. A `define` edge connects a definition node to the statement that created it, which is how a slice traversal crosses a def-use chain. A `source-to-sink` edge lies on a reported flow. Because each edge stores its justification, the interface can display why an edge exists, and the graph statistics are measured rather than estimated.

Slicing is reachability over that graph. A backward slice from a statement or from a finding's sink follows incoming data, control, call, return, taint and sanitise edges; a forward slice follows the same edges outwards. Each included statement is recorded with the reason it was reached. Slices are conservative by design - they include every statement the may-analysis cannot exclude - and the slice carries a note saying that it is not minimal. Slice size is capped at 5,000 statements and 500 listed edges, and a slice that hits a cap says so.

3.3.9 The rule language

Rules are data, not code. A rule file declares `version`, then a sequence of `source`, `sink`, `sanitiser`, `validator` and `flow` blocks with attributes such as `function`, `returns` or `arguments`, `context`, `category`, `severity`, `description`, `sources`, `sinks` and `sanitisers`. The file is parsed by the rule-language grammar, which is compiled by the same LR generator as CT-C, and the parsed declarations are then validated declaratively: an unknown reference, a missing context, a duplicate identifier, an argument index on a sink with no declared arguments, or a flow that names a rule that does not exist produces a diagnostic with a code, a line and a column. A rule file with any error is refused before analysis, so the engine never runs with a partially loaded rule set.

The built-in rule file declares 5 sources, 8 sinks, 5 sanitisers, 2 validators and 4 flows, covering command injection, SQL injection, path traversal, format-string misuse and the corresponding sanitisers and validators. The rule set is fingerprinted; the fingerprint appears in the health endpoint, in every report and in every finding's evidence block, so a result can be tied to the exact rule configuration that produced it.

CHAPTER 4
IMPLEMENTATION DETAILS

4.1 Technology and Application Workflow

The engine is implemented in Python 3.11 and imports nothing outside the standard library: `http.server` provides the HTTP server, `json` the wire format, `unittest` the test framework, `hashlib` the finding and rule fingerprints, `dataclasses` the analysis records and `re` and `os` the routing and static-file checks. There is no `requirements.txt` because there is no requirement beyond the interpreter. The interface is HTML, one stylesheet and plain ES modules; it has no build step, no framework and no runtime dependency, and its tests run on the Node built-in test runner against a stub DOM.

A local session is started with `python3 -m codetrace.api`, which serves the interface and the API on port 8000, or with `scripts/serve.sh --port 9000`, which passes its arguments through. `scripts/tests.sh` runs the engine suite and then the interface suite; `scripts/evaluate.sh` runs the labelled corpus and writes JSON evidence; `scripts/audit.py` performs an independent audit of the repository; `scripts/mutation_check.py` breaks the engine on purpose and checks that the suite notices. No command needs network access, a key or a service account.

The HTTP API is small and explicit. `POST /api/analyze` accepts a source string, an optional filename, an optional rule file, a flag that selects whether the built-in rules are extended, a flag that enables or disables the dependency graph, and a parser mode of `lalr`, `slr` or `canonical`. `POST /api/rules/validate` parses and validates a rule file without analysing anything. `POST /api/compare` compares two runs still held in memory. The metadata endpoints are `GET /api/health`, `/api/schemas`, `/api/language`, `/api/grammar`, `/api/rules`, `/api/examples` and `/api/examples/{id}`. A stored run is addressed by `GET /api/runs/{id}` and its sub-resources: findings (with filters), one finding, a finding's slice, the AST, a control-flow graph, a data-flow result for a named function and analysis, one statement, the dependency graph and the exported report in JSON or HTML.

Validation happens before any analysis work. The request body is limited to 1,100,000 bytes, the source to 524,288 bytes and a submitted rule file to the same size; an unknown parameter is rejected with `400 invalid-parameter`, an unsupported parser mode with `400 invalid-parser-mode`, a broken rule file with `422 invalid-rules`, and an oversized body with `413`. Every error uses one envelope with a stable code, a message that quotes the offending value and a detail field that never contains a stack trace or a server path. The engine additionally caps the number of functions, data-flow iterations, graph nodes and edges, slice size, parse steps and syntax errors, and recursion depth; when a cap is reached the run reports which one, sets `limits_hit` and marks the analysis partial.

4.2 Worked Example

The example used here is the bundled `examples/vulnerable/command_injection.ctc`, reproduced below exactly as it appears in the repository, comments included.

```
// expect: 1 finding, rule command_execution, HIGH, category command-injection
// Untrusted standard input is concatenated into a shell command.
string build_command(string user_input) {
  return "ls " + user_input;
}

int main() {
  string name = input();
  string command = build_command(name);
  system(command);
  return 0;
}
```

The processing path is the pipeline of Chapter 3 applied to this file. Lexing produces 43 tokens over 13 lines and 330 bytes, with no lexical diagnostics. Parsing uses the LALR tables, builds 22 AST nodes and recovers from no error, so the run is accepted. Semantic analysis creates 7 symbols in 3 scopes, resolves 3 calls and reports no diagnostic: `input` and `system` are resolved as external functions declared by the rule set, and `build_command` as a user function. Statement numbering yields 5 statements: `return "ls " + user_input;` in `build_command` and, in `main`, the declaration of `name`, the declaration of `command`, the call to `system` and the final return.

Control-flow construction produces 4 basic blocks and 2 typed edges, with no unreachable block, and numbers 3 definitions. The three data-flow analyses converge in 12 solver iterations in total. Taint analysis runs the intraprocedural analysis for both functions, computes the summary for `build_command` - parameter 0 influences the returned value - and then runs the analysis again with the summary available; the argument `name` reaches `command` through the call and then reaches argument 0 of `system`, which the rule `command_system` declares as a sink in the `command` context. The resulting state carries one atom, `src:stdin@command`. The dependency graph has 10 nodes and 7 edges, and finding construction emits exactly one finding.

The finding carries identifier `FEF13A28F7E0E`, produced by hashing the rule identifier, the sink function, the sink rule, the argument index, the contributing source rules and the whitespace-normalised text of the sink statement. It is classified `HIGH` severity, category `command-injection`, certainty `approximate` and feasibility `unconditional`, and it is marked interprocedural because the path crosses a function boundary. Its evidence path has eight entries, in this order:

1. `input()` introduces untrusted data under source rule `stdin`, at line 8, column 17 of `main`.
2. `name` is declared and initialised with a tainted value at line 8.
3. Argument 0 of the call to `build_command` is tainted at line 9, column 34.
4. `user_input`, parameter 0 of `build_command`, receives the tainted value at line 3, column 22.
5. The return statement of `build_command` at line 4 returns a tainted value.
6. The call to `build_command` at line 9 propagates taint to the caller through the function summary.
7. `command` is declared and initialised with a tainted value at line 9.
8. Argument 0 of `system` consumes data that is not safe for context `command`, at line 10, column 10.

The explanation text assembled from these steps states the source line, the sink line, the atom that was propagated, the seven propagation steps between the source and the sink, the three sanitisers that the flow rule lists (`shell_escape`, `parse_to_int` and `validate_command`) and the fact that none of them removed the atoms that reached the sink. Two limitations are attached: the interprocedural reasoning is context-insensitive, which can add flows that no single call site can create, and the finding

is static, so exploitability was not established. The remediation hint advises against building a command from untrusted input and suggests passing arguments to a fixed program instead. The record ends with an evidence block naming the engine version `1.0.0`, the rule-set fingerprint `builtin-3387b7dcd97a95f3`, the atom list and the statement span of the sink. A short excerpt of the exported finding is:

```
"rule_id": "command_execution",
"category": "command-injection",
"severity": "HIGH",
"certainty": "approximate",
"sink": {"function": "system", "argument_index": 0,
         "line": 10, "col": 10, "statement_text": "system(command);"},
"source": {"rule": "stdin", "function": "input", "line": 8, "col": 17},
"sanitizers_seen": [],
"branch_conditions": [],
"feasibility": "unconditional",
"interprocedural": true
```

The complementary case establishes what the analysis does with a negative result. In `examples/safe/source_never_reaches_sink.ctc` the value read from `input()` is overwritten with the constant string `"whoami"` before `system(value)` is called. The reaching-definitions result shows the constant definition as the only definition reaching the sink statement, the taint state at that statement is empty, and the run reports no finding and no diagnostic. The sink call is nevertheless recorded as inspected with the disposition "no taint reached it", so the absence of a finding is an accounted result rather than an unexamined silence.

Malformed input produces the third kind of evidence. In `examples/syntax_errors/missing_semicolon.ctc` the declaration on line 3 is not terminated, and the parser reports `SYN001` at 4:3 for the unexpected identifier `system` with the list of expected tokens, followed by a second `SYN001` at 4:15 for the unexpected semicolon. The run is marked partial with the reason that the parser recovered from two syntax errors, so statements were skipped. For a construct outside the subset the lexical phase contributes first: `examples/unsupported/switch_statement.ctc` produces `LEX100` at 4:3 and 5:5, `LEX102` for the colon at 5:11 and `SYN100` at 4:3, and the partial-analysis reason lists the constructs that were not analysed.

What this evidence establishes is specific: for this file, with this rule set and this engine version, the recorded path connects a declared source to a declared sink through a named chain of program points, and the data-flow facts that justify each hop are available from the API. It does not establish that the program is exploitable, that the path is the only one, or that the analysis would find an equivalent flow written differently - for example through a pointer, through an array element with a computed index, or through a value chosen by a branch on untrusted data. Those are exactly the cases the limitation list names.

4.3 Testing and Verification Evidence

Verification uses four independent instruments, and the results below were observed on this checkout on 19 October 2026. The engine suite is run with `python3 -m unittest discover -s tests -t .`; it executes 350 tests covering the lexer, the parser and its tables, the grammar theory, semantic analysis, control-flow graphs, the three data-flow analyses, taint propagation, rules, slicing, the API over real HTTP, reporting and the end-to-end workflow. The run reported `Ran 350 tests` and `OK`. The interface suite is run with `npm test` in `frontend/`; it executes 53 tests against a stub DOM and a real engine over HTTP, and reported 53 passing and 0 failing.

The labelled corpus is the third instrument. It consists of 33 `.ctc` files, each carrying a `// expect:` header written before the analysis is run, and a separate `expectations.json` that records the label, the expected rules, categories, severities and diagnostics, and a one-sentence rationale per case. `python3 evaluation/run_evaluation.py` reported 20 true positives, 0 false positives and 0 false negatives at the finding level, 9 of 9 file-level true negatives over the safe and sanitised cases, and precision, recall and F1 of 1.0, over 33 files in 0.974 seconds. The corpus is deliberately partitioned: 18 files carry finding-level expectations, 9 are file-level negative cases and 6 are diagnostic-only cases that must produce a named diagnostic and no finding; a program rejected before analysis is not counted as a true negative.

The repository audit is the fourth instrument. `python3 scripts/audit.py` reported "all 69 checks passed". Its checks are not a restatement of the unit tests: they include an independent computation of the LR tables' shape, hand-computed data-flow results compared against the engine, checks that every documented file exists and every cross-reference resolves, checks that the module table in the design document matches the tree, checks that the rule catalogue covers the pipeline and that a validator-guarded flow leaves a suppression record rather than silence, and checks that a program which would create a canary file when executed leaves no such file. A separate mutation harness holds 18 deliberate defects across the parser, semantics, control-flow construction, data flow, taint, slicing, rules and reporting; each defect names the tests that must notice it, the harness restores the file byte for byte, and it refuses to start when a target file has uncommitted changes. Run on this checkout, `python3 scripts/mutation_check.py` reported all 18 defects caught, which is the evidence that the suite can fail rather than merely passing.

Beyond the four instruments, the claims made in this report about specific artefacts were re-checked directly against the implementation before they were written. The grammar and rule-language figures were produced by building the tables during the drafting of the report, the worked example's metrics, finding identifier, evidence path and disposition records were produced by calling `analyse_source` on the bundled file, and the diagnostic excerpts were produced by analysing the malformed and unsupported examples. The suite count of 350 is itself measured: it is a parameterised test population assembled at run time, which is one reason a figure quoted from an older document can lag behind the code.

Three qualifications belong with these numbers. First, the passing suite shows that the implementation agrees with the tests, not that either is correct in an absolute sense; the independent audit and the hand-computed expectations are what give the agreement evidential weight. Second, the corpus is small and hand-written by the project authors, so precision and recall of 1.0 describe 33 files and say nothing about real-world detection quality; a single additional case would move the figures. Third, test counts and timings are checkout- and machine-specific. The verification table in `README.md` records the same 350 engine tests and 53 interface tests; these figures should be refreshed from the commands rather than treated as a permanent benchmark.

4.4 Current Progress and Challenges

All the stages described in Chapter 3 are implemented in the checkout and exercised by the suites: lexer, grammar analysis and table construction, parsing with recovery, AST construction, semantic analysis, control-flow graphs, the data-flow framework and its three instantiations, the rule language and its validation, taint analysis with summaries and guard suppressions, the dependency graph, slicing, finding construction and explanation, JSON and HTML reporting, the HTTP API and the nine-page interface. The generated grammar report and the language description served by the API are produced from the implementation, so the documentation cannot describe a grammar the parser does not run.

Several capabilities are implemented only in part, and the report states them as such. Interprocedural reasoning uses context-insensitive summaries, so a parameter tainted at one call site is treated as tainted at every call site of that function, and the finding that results is labelled approximate. Path reasoning recognises literal conditions and declared validator guards but does not solve correlated conditions, so `if (a)` followed by `if (a)` is not related. Loop trip counts are not computed, so a value tainted before a loop whose body always overwrites it is still reported. Implicit flows are not modelled. Arrays are tracked as whole objects rather than element by element, and pointers, structures and multiple translation units are outside the language. Finding identity is derived in part from the text of the sink statement, so renaming the variable at a sink reports the finding as new and resolved rather than moved when two runs are compared.

The main engineering challenges were the ones the design implies. Source locations had to survive every transformation, because a finding is only traceable if its line and column refer to the original text; the solution was to carry spans on AST nodes and to address statements and definitions by identifier thereafter. Symbol identity had to survive shadowing and the merge of scopes at block exits, which is why every declaration receives a number at the point where it is created rather than a name that can be reused. Taint had to distinguish "unsafe for a command" from "unsafe for SQL", which is what the atom-and-context design provides. Finally, the analysis had to disclose its own silences: the disposition records and the guard-suppression records exist because a finding list, on its own, cannot distinguish "checked and clean" from "not checked".

The remaining work is therefore not a list of missing stages but a list of precision improvements. The most valuable are context-sensitive summaries, path conditions that survive across branches, element-level array tracking, and alias analysis for a language extension that admits pointers. Each of these would change the certainty labelling of existing findings, which is why the finding record treats certainty and limitations as first-class fields rather than as prose.

CONCLUSION AND FUTURE ENHANCEMENTS

CodeTrace demonstrates how compiler representations support an explainable static program and security analysis. A hand-written lexer and a canonical LR(1) parser generator built from the project's own grammar produce a located syntax tree; scoped symbol tables give every declaration a stable identity; structured control-flow graphs and a shared worklist framework compute reaching definitions, live variables and definite assignment; a declarative rule language supplies sources, sinks, context-specific sanitisers and validator guards; and a taint analysis over atoms and contexts turns those ingredients into findings that carry an evidence path, a certainty, a limitation list and a remediation hint. The dependency graph and the slices expose the same result structurally, and the HTTP API and browser interface make every intermediate artefact inspectable.

The practical significance of the work is twofold. As a compiler-design artefact it shows the stages composing into a whole-program analysis, with the tables, the trees, the graphs and the data-flow states available for inspection rather than hidden behind a command-line result. As a security-analysis artefact it shows what an honest static tool must disclose: that findings are potential flows and not exploits, that a clean run is bounded by the constructs and rules that were considered, and that the approximations which produce false positives and false negatives can be named.

Future work should prioritise precision before coverage. Context-sensitive function summaries and summary edges in the dependency graph would remove the largest source of approximation in the current findings. Path conditions carried across branches would let the engine relate correlated tests and reason about loop bodies instead of treating them as possibly skipped. Element-level array tracking and an alias analysis for a pointer extension would widen the language without pretending that full C is supported. On the reporting side, a comparison view that explains *why* a finding moved, and a machine-readable certificate of what was and was not analysed, would extend the existing disposition records. Finally, an independent evaluation on programs written by someone other than the authors, and a comparison against an established analyser on that corpus, would replace the present regression-grounded claim with a measured one.

REFERENCES

[1] A. V. Aho, M. S. Lam, R. Sethi, and J. D. Ullman, *Compilers: Principles, Techniques, and Tools*, 2nd ed. Boston, MA, USA: Pearson/Addison-Wesley, 2006.

[2] D. E. Knuth, "On the translation of languages from left to right," *Information and Control*, vol. 8, no. 6, pp. 607-639, 1965.

[3] F. L. DeRemer, "Simple LR(k) grammars," *Communications of the ACM*, vol. 14, no. 7, pp. 453-460, 1971.

[4] G. A. Kildall, "A unified approach to global program optimization," in *Proceedings of the ACM Symposium on Principles of Programming Languages (POPL)*, 1973, pp. 194-206.

[5] P. Cousot and R. Cousot, "Abstract interpretation: a unified lattice model for static analysis of programs by construction or approximation of fixpoints," in *Proceedings of the ACM Symposium on Principles of Programming Languages (POPL)*, 1977, pp. 238-252.

[6] F. Nielson, H. R. Nielson, and C. Hankin, *Principles of Program Analysis*. Berlin, Germany: Springer, 1999.

[7] M. Weiser, "Program slicing," *IEEE Transactions on Software Engineering*, vol. SE-10, no. 4, pp. 352-357, 1984.

[8] J. Ferrante, K. J. Ottenstein, and J. D. Warren, "The program dependence graph and its use in optimization," *ACM Transactions on Programming Languages and Systems*, vol. 9, no. 3, pp. 319-349, 1987.

[9] S. Horwitz, T. Reps, and D. Binkley, "Interprocedural slicing using dependence graphs," *ACM Transactions on Programming Languages and Systems*, vol. 12, no. 1, pp. 26-60, 1990.

[10] T. Reps, S. Horwitz, and M. Sagiv, "Precise interprocedural dataflow analysis via graph reachability," in *Proceedings of the ACM Symposium on Principles of Programming Languages (POPL)*, 1995, pp. 49-61.

[11] B. Chess and G. McGraw, "Static analysis for security," *IEEE Security & Privacy*, vol. 2, no. 6, pp. 76-79, 2004.

[12] V. B. Livshits and M. S. Lam, "Finding security vulnerabilities in Java applications with static analysis," in *Proceedings of the 14th USENIX Security Symposium*, 2005, pp. 271-286.

[13] E. J. Schwartz, T. Avgerinos, and D. Brumley, "All you ever wanted to know about dynamic taint analysis and forward symbolic execution (but might have been afraid to ask)," in *Proceedings of the IEEE Symposium on Security and Privacy*, 2010, pp. 317-331.

[14] B. Johnson, Y. Song, E. Murphy-Hill, and R. Bowdidge, "Why don't software developers use static analysis tools to find bugs?" in *Proceedings of the 35th International Conference on Software Engineering (ICSE)*, 2013, pp. 672-681.

[15] C. Sadowski, E. Aftandilian, A. Eagle, L. Miller-Cushon, and C. Jaspan, "Lessons from building static analysis tools at Google," *Communications of the ACM*, vol. 61, no. 4, pp. 58-66, 2018.

[16] MITRE, "Common Weakness Enumeration (CWE)," including CWE-22, CWE-78, CWE-89 and CWE-134. [Online]. Available: https://cwe.mitre.org/